
Building an Application

Now that you have learned the basics of the Clojure language, it is time for you to begin using Clojure in your own projects. But when you run out the door to start work on your killer Clojure app, you quickly discover that language knowledge is only part of what you need to work effectively in an ecosystem. You have questions like these:

- What tools do I use to organize projects and dependencies?
- What is a good workflow for writing code?
- How do I make sure that my code is correct?
- How do I keep code flexible and maintainable?
- What libraries do I need?
- How do I put Clojure on the Web?

There will never be a single, one-size-fits-all answer to these questions. Clojure runs on the JVM, a huge ecosystem where hundreds of approaches have flourished. But you have to start somewhere. In this chapter, we will give you temporary answers to these questions. These answers have worked well for us, and you can use them for a short while. As your own preferences and sensibilities evolve, you can adapt or abandon the approaches in this chapter in favor of approaches that work best for you.

As our sample application, we will implement a web version of the Clojurebreaker game. In Clojurebreaker, a code-maker (the program) creates a secret code of N-ordered colored pegs. A code-breaker (the human player) then submits a guess. The code-maker scores the guess as follows:

- One black peg for each peg of the right color in the right position
- One white peg for each peg of the right color but not in the right position

The game ends with a correct guess or after reaching some predetermined limit on the number of guesses.

While we will show you all the code as we go, this chapter is not really about code. It is about a style of attacking problems and about the details of delivering solutions within the Clojure ecosystem. Let's get started.

10.1 Scoring a Clojurebreaker Game

As a Clojure programmer, one question you will often ask is, “Where do I need state to solve this problem?” Or, better yet, “How much of this problem can I solve *without* using any state?”

With Clojurebreaker (and with many other games), the game logic itself is a pure function. It takes a secret and a guess and returns a score. Identifying this fact early gives us two related advantages:

- The score function will be trivial to write and test in isolation.
- We can comfortably proceed to implement score without even thinking about how the rest of the system will work.

Scoring itself divides into two parts: tallying the exact matches and tallying the matches that are out of order. Each of these parts can be its own function. Let's start with the exact matches. To make things concrete, we will pick a representation for the pegs that facilitates trying things at the REPL: the four colors `:r` (red), `:g` (green), `:b` (blue), and `:y` (yellow). The function will return the count of exact matches, which we can turn into black pegs in a separate step later. Here is the shell of the function we think we need:

```
clojurebreaker/snippets.clj
(defn exact-matches
  "Given two collections, return the number of positions where
  the collections contain equal items."
  [c1 c2])
```

Hold the phone—that doc string doesn't say anything about games or colors or keywords. What is going on here? While some callers (e.g., the game) will eventually care about the representation of the game state, `exact-matches` doesn't need to care. So, let's keep it generic. A key component of responsible Clojure design is to think in data, rather than pouring object concrete at every opportunity.

When described as a generic function of data, `exact-matches` sounds like a function that might already exist. After searching through the relevant namespaces (`clojure.core` and `clojure.data`), we discover that the closest thing to `exact-matches` is `clojure.data's diff`. `diff` recursively compares two data structures, returning a three-tuple of things-in-a, things-in-b, and things-in-both. The

things-in-both is nothing other than the exact matches we are looking for. Try it at the REPL:

```
(require '[clojure.data :as data])
(data/diff [:r :g :g :b] [:r :y :y :b])
-> [[nil :g :g] [nil :y :y] [:r nil nil :b]]
```

The non-nil entries in `[:r nil nil :b]` are the exact matches when comparing `r/g/g/b` and `r/y/y/b`. With `diff` in hand, the implementation of exact-matches is trivial:

`clojurebreaker/src/clojurebreaker/game.clj`

```
(defn exact-matches
  "Given two collections, return the number of positions where
  the collections contain equal items."
  [c1 c2]
  (let [[_ _ matches] (data/diff c1 c2)]
    (count (remove nil? matches))))
```

Again, we test at the REPL against an example input:

```
(exact-matches [:r :g :g :b] [:r :y :y :b])
2
```

Now let's turn our attention to the unordered matches. To calculate these, we need to know how many of each colored peg are in the secret and in the guess. This sounds like a job for the frequencies function:

```
(def example-secret [:r :g :g :b])
(frequencies example-secret)
```

```
-> {:r 1, :g 2, :b 1}
```

```
(def example-guess [:y :y :y :g])
(frequencies example-guess)
```

```
-> {:y 3, :g 1}
```

To turn those two frequencies into the unordered-matches, we need to do two additional things:

- Consider only the keys that are present in both the secret and the guess
- Count only the overlap (i.e., the minimum of the vals under each key)

Again, we hope these operations already exist, and happily they do. You can keep the keys you need with `select-keys`:

```
(select-keys (frequencies example-secret) example-guess)
-> {:g 2}
```

```
(select-keys (frequencies example-guess) example-secret)
-> {:g 1}
```

And you can count the overlap between two frequency maps using `merge-with`:

```
(merge-with min {:g 1} {:g 2})
-> {:g 1}
```

Combining frequencies and `select-keys` and `merge-with` gives the following definition for `unordered-matches`:

```
clojurebreaker/src/clojurebreaker/game.clj
```

```
(defn unordered-matches
  "Given two collections, return a map where each key is an item
  in both collections, and each value is the number of times the
  value occurs in the collection with fewest occurrences."
  [c1 c2]
  (let [f1 (select-keys (frequencies c1) c2)
        f2 (select-keys (frequencies c2) c1)]
    (merge-with min f1 f2)))
```

which, of course, we should verify at the REPL:

```
(unordered-matches [:r :g :g :b] [:y :y :y :g])
-> {:g 1}
```

That's nice, with one subtlety. `unordered-matches` counts matches *regardless* of order, while the game will want to know only the matches that are *not* in the right order. Even though the game doesn't seem to need `unordered-matches`, writing it was a win because of the following:

- `unordered-matches` does exactly one thing. To write a not-ordered match, we would have to reimplement `exact-matches` inside `unordered-matches`.
- The two simple functions we just wrote are exactly the functions we need to compose together to get the not-ordered semantics. Just subtract the results of `exact-matches` from the results of `unordered-matches`.

With the two primitives in place, the score operation simply compounds them:

```
clojurebreaker/src/clojurebreaker/game.clj
```

```
(defn score
  [c1 c2]
  (let [exact (exact-matches c1 c2)
        unordered (apply + (vals (unordered-matches c1 c2)))]
    {:exact exact :unordered (- unordered exact)}))
```

And the REPL rejoices:

```
(score [:r :g :g :b] [:r :y :y :g])
-> {:exact 1, :unordered 1}
```

At this point, we have demonstrated a partial answer to the question, “What is a good workflow for writing code?” In summary:

- Break apart the problem to identify pure functions.
- Learn the standard library so you can find functions already written.
- Pour no concrete (use data as data).
- Test inside out from the REPL.

In our experience, programmers trying this workflow for the first time make two typical mistakes:

- Coding too much
- Complicating the tests

You have written too much code whenever you don't understand the behavior of a form, but you haven't yet tested and understood all of its subforms. Many developers have an intuition of "write *X* lines of code and then test," where *X* is the smallest number of lines that can do something substantial. In Clojure, *X* is significantly smaller than one, which is why we emphasize building functions inside out at the REPL.

"Complicating the tests" is more subtle, and we will take it up in the next section.

10.2 Testing the Scorer

In the previous section, we developed the score function iteratively at the REPL and saw it work correctly with a few example inputs. It doesn't take much commitment to quality to want to do more validation than that! Let's begin by teasing apart some of the things that people mean when they say "testing." Testing includes the following:

- Thinking through whether the code is correct
- Stepping through the code in a development environment where you can see everything that is happening
- Crafting inputs to cover the various code paths
- Crafting outputs to match the crafted inputs
- Running the code with various inputs
- Validating the results for correctness
- Automating the validation of results
- Organizing tests so that they can be automatically run for regression purposes in the future

This is hardly an exhaustive list, but it suffices to make the point of this section. In short, testing is often complex, but it can be simple.

Traditional unit-testing approaches compect many of the testing tasks listed earlier. For example, input, output, execution, and validation tend to be woven together inside individual test methods. On the other hand, the minimal REPL testing we did before simply isn't enough. Can we get the benefit of some of the previous ideas of testing, without the complexity of unit testing? Let's try.

Crafting Inputs

We have already seen the score function work for a few handcrafted inputs. How many inputs do we need to convince ourselves the function is correct? In a perfect world, we would just test all the inputs, but that is almost always computationally infeasible. But we are lucky in that the problem of scoring the game is essentially the same for variants with a different number of colors or a different number of pegs. Given that, we actually can generate *all* possible inputs for a small version of the game.

The branch of math that deals with the different ways of forming patterns is called *enumerative combinatorics*. It turns out that the Clojure library `math.combinatorics` has the functions we need to generate all possible inputs. Add the following form under the `:dependencies` key in your `project.clj` file, if it is not already present:

```
[org.clojure/math.combinatorics "0.0.1"]
```

The `selections` function takes two arguments (a collection and a size), and it returns every structure of that size made up of elements from that collection. Try it for a tiny version of Clojurebreaker with only three colors and two columns:

```
(require '[clojure.math.combinatorics :as comb])
(comb/selections [:r :g :b] 2)
-> ((:r :r) (:r :g) (:r :b)
    (:g :r) (:g :g) (:g :b)
    (:b :r) (:b :g) (:b :b))
```

So, `selections` can give us a possible secret or a possible guess. What about generating inputs to the score function? Well, that is just selecting two selections from the selections:

```
(-> (comb/selections [:r :g :b] 2)
    (comb/selections 2))
-> (81 pairs of game positions omitted for brevity)
```

Let's put that into a named function:

```
clojurebreaker/src/clojurebreaker/game.clj
(defn generate-turn-inputs
  "Generate all possible turn inputs for a clojurebreaker game"
```

```

    with colors and n columns"
[colors n]
(-> (comb/selections colors n)
    (comb/selections 2)))

```

All right, inputs generated. We are going to skip thinking about outputs (for reasons that will become obvious in a moment) and turn our attention to running the scorer with our generated inputs.

Running a Test

We are going to write a function that takes a sequence of inputs and reports a sequence of inputs and the result of calling `score`. We don't want to commit (yet) to how the results of this test run will be validated. Maybe a human will read it. Maybe a validator program will process the results. Either way, a good representation of each result might be a map with the keys `secret`, `guess`, and `score`.

All this function needs to do is call `score` and build the collection of responses:

```

clojurebreaker/src/clojurebreaker/game.clj
(defn score-inputs
  "Given a sequence of turn inputs, return a lazy sequence of
  maps with :secret, :guess, and :score."
  [inputs]
  (map
   (fn [[secret guess]]
     {:secret (seq secret)
      :guess (seq guess)
      :score (score secret guess)})
   inputs))

```

Try it at the REPL:

```

(->> (generate-turn-inputs [:r :g :b] 2)
      (score-inputs))
-> ({:secret (:r :r), :guess (:r :r),
      :score {:exact 2, :unordered 0}}
   {:secret (:r :r), :guess (:r :g),
      :score {:exact 1, :unordered 0}}
   ;; remainder omitted for brevity

```

If a human is going to be reading the test report, you might decide to format a text table instead, using `score print-table`. While we are at it, let's generate a bigger game (four colors by four columns) and print the table to a file:

```

(use 'clojure.pprint)
(require '[clojure.java.io :as io])
(with-open [w (io/writer "scoring-table")]
  (binding [*out* w]

```

```
(print-table (->> (generate-turn-inputs [:r :g :b :y] 4)
                (score-inputs))))
-> nil
```

If you look at the scoring-table file, you should see 65,536 different secret/guess combinations and their scores.

Validating Outputs

At this point, it is obvious why we skipped crafting the outputs. The program has done that for us. We just have to decide how much effort to spend validating them. Here are some approaches we might take:

- Have a human code-breaker expert read the entire output table for a small variant of the game. This has the advantage of being exhaustive but might miss logic errors that show up only in a larger game.
- Pick a selection of results at random from a larger game and have a human expert verify that.

Because the validation step is separated from generating inputs and running the program, we can design and write the various steps independently, possibly at separate times.

Moreover, the validator knows nothing about how the inputs were generated. With unit tests, the inputs and outputs come from the same programmer's brain at about the same time. If that programmer is systematically mistaken about something, the tests simply encode mistakes as truth. This is not possible when the outputs to validate are chosen exhaustively or randomly.

We will return to programmatic validation later, but first let's turn to regression testing.

Regression Testing

How would you like to have a regression suite that is more thorough than the validation effort you have made? No problem.

- Write a program whose results should not change.
- Run the program once, saving the results to a (well-named!) file.
- Run the program again every time the program changes, comparing with the saved file. If anything is different, the program is broken.

The nice thing about this regression approach is that it works even if you never did *any* validation of results. Of course, you should still do validation, because it will help you narrow down where a problem happened. (With no

validation, the regression error might just be telling you that the old code was broken and the new code fixed it.)

How hard is it to write a program that should produce exactly the same output? Call only pure functions from the program, which is exactly what our score-inputs function does.

Wiring this kind of regression test into a continuous integration build is not difficult. If you do it, think about contributing it to whatever testing framework you use.

Now we have partially answered the question, “How do I make sure that my code is correct?” In summary:

- Build with small, composable pieces (most should be pure functions).
- Test forms from the inside out at the REPL.
- When writing test code, keep input generation, execution, and output validation as separate steps.

This last idea is so important that it deserves some library support. So, before we move on, we are going to introduce `test.generative`, a library that aspires to bring simplicity to testing.

10.3 test.generative

The `test.generative` library divides testing into three key steps:

- Generating test inputs
- Invoking test functions
- Validating results

Each of these three steps is implemented via functions, which are then composed into tests via the `defspec` form.

Let’s install `test.generative` and take it for a spin. Add the following line to your `project.clj`:

```
[org.clojure/test.generative "0.1.3"]
```

Now execute whatever steps you use to reload project code and meet us at the REPL to generate some test data.

Generating Data

From the REPL, require the `generators` namespace as follows:

```
(require '[clojure.test.generative.generators :as gen])
```

The `generators` namespace contains functions to generate pseudorandom values for different common datatypes. For example, for every Java primitive, there is a generator function with the same name. Try a few of them:

```
(gen/int)
-> 977378563
```

```
(gen/char)
-> \A
```

```
(gen/boolean)
-> true
```

Note that your results will likely not be identical to those shown earlier, since the point of these values is to be random.

You can also generate random values for different Clojure collection types. These functions are parameterized by the type of collection to make. For example:

```
(gen/vec gen/boolean)
-> [false false true true true false]
```

```
(gen/hash-map gen/byte gen/int)
-> {32 -504310803, 100 424501842, 5 1439482147, 37 1161641068}
```

In addition to the basic types, you can use several knobs to control how the types are generated. You can choose a probability distribution:

```
(gen/geometric 0.02)
-> 10
```

And you can control how collections are sized, with either a constant or a distribution:

```
(gen/list gen/int 2)
-> (-1029960512 1985289448)
```

```
(gen/list gen/int (gen/uniform 0 5))
-> (315829211)
```

There are several other goodies, but we won't spoil all the fun of discovery. Do a `dir` of the namespace, and familiarize yourself with the other standard generators that are available.

To test the Clojurebreaker scorer, we will want a function that generates a random secret (or guess). Sticking with the keyword representation for peg colors, such a function could be as follows:

```
clojurebreaker/test/clojurebreaker/game_test.clj
(defn random-secret
  []
  (gen/vec #(gen/one-of :r :g :b :y) 4))
```

Don't forget to try `random-secret` at the REPL before moving on to think about validation.

Programmatic Validation

Programmatic validation should be more than just a hand-coded spot-check of specific values. Such spot-checks are a poor use of time when we can easily review a set of outputs for validity and then save them as a regression test. Instead, programmatic validators should enforce logical invariants about the input and output data.

Here are some invariant properties of the Clojurebreaker scoring function:

- Scoring is symmetric. If you invert the secret and guess arguments, the score should be the same.
- The sum of the exact and unordered matches must be greater than or equal to zero and less than or equal to the number of pegs.
- If you scramble the order of pegs in a guess, the sum of exact and unordered matches in the score will not change.

Let's encode each of these invariants as a boolean function of secret, guess, and score:

```
clojurebreaker/test/clojurebreaker/game_test.clj
(ns clojurebreaker.game-test
  (:use [clojure.test.generative :only (defspec) :as test])
  (:require [clojure.test.generative.generators :as gen]
            [clojurebreaker.game :as game]
            [clojure.math.combinatorics :as comb]))

(defn matches
  "Given a score, returns total number of exact plus
  unordered matches."
  [score]
  (+ (:exact score) (:unordered score)))

(defn scoring-is-symmetric
  [secret guess score]
  (= score (game/score guess secret)))

(defn scoring-is-bounded-by-number-of-pegs
  [secret guess score]
  (< 0 (matches score) (count secret)))
```

```
(defn reordering-the-guess-does-not-change-matches
  [secret guess score]
  (= #{(matches score)}
     (into #{} (map
                #(matches (game/score secret %))
                (comb/permutations guess))))))
```

While we often type code at the REPL and then paste it into the appropriate file, we went the other way with the invariant functions earlier, adding them to the source file first. But this doesn't mean we have to lose the immediacy of the REPL. From the REPL, we can reload the namespace and then move into the namespace with `in-ns`:

```
(require :reload 'clojurebreaker.game-test)
(in-ns 'clojurebreaker.game-test)
```

Now that we are in the namespace we want to use, we have access to all of its public names and aliases. This makes it convenient to handcraft some sample data as a quick sanity check for the validation functions:

```
(def secret [:r :g :g :b])
(def guess [:r :b :b :y])

(scoring-is-symmetric secret guess
  (game/score secret guess))
-> true

(scoring-is-bound-by-number-of-pegs
  secret guess (game/score secret guess))
-> true

(reordering-the-guess-does-not-change-matches
  secret guess (game/score secret guess))
-> true
```

With a test data generator and some validation functions in hand, we are now ready to wire everything together in a `defspec`.

defspec

The `defspec` takes three required arguments:

- A name for the spec
- The function to test
- The arguments to generate

After the arguments come zero or more body forms, which perform validations. Here is a trivial example:

```
clojurebreaker/snippets.clj
(defspec closed-under-addition
  '+'
  [^long a ^long b]
  (assert (integer? %)))
```

The arguments look like normal Clojure arguments, but there is a twist. Normal function arguments can optionally be tagged with type hints. Spec arguments *must* be tagged, with hints that tell how to *generate* the arguments.

The `test.generative` function `generate-test-data` takes an argument spec and generates an infinite sequence of test data. You can call `generate-test-data` directly to generate arguments separately from any test execution:

```
(take 3
  (test/generate-test-data '[long long]))

-> ([-5025562857975149833 -5843495416241995736]
   [5694868678511409995 5111195811822994797]
   [-6169532649852302182 -1782466964123969572])
```

The body of a `defspec` has access to the arguments by name, just like a normal function. In addition, it also has access to `%`, which holds the return values from calling the test function. In the `closed-under-addition` example, the single validator form checks that the result of addition is an integer, without even bothering to look at the input arguments:

```
(assert (integer? %))
```

More sophisticated validators will typically refer to both the input arguments and the result.

Running Tests

Specs are still functions, so you can run them by simply invoking the function. If you evaluated the `closed-under-addition` spec at the REPL while reading the previous section, you can invoke it now, passing in some integers:

```
(closed-under-addition 1 2)
-> nil
```

But the more interesting thing is to run the spec with generated inputs. There are three helpers in `test.generative` for this purpose:

- `test-vars` takes one or more vars and runs the specs referenced by the vars.
- `test-dirs` takes one or more directories and reflectively finds all specs in those directories and runs them.

- `test-namespaces` takes on more namespaces and reflectively finds all specs in those namespaces and runs them.

Let's run the `closed-under-addition` spec:

```
(require '[clojure.test.generative :as test])
(test/test-vars #'closed-under-addition)

-> [#<core$future_call$reify__5684@603a3e21: :pending>
    #<core$future_call$reify__5684@fc519e2: :pending>]
```

The call to `test-vars` returns some number of futures (two in the earlier output). Then nothing happens for about ten seconds, followed by REPL output similar to this:

```
{:iterations 179766,
 :msec 10228,
 :var #'clojurebreaker.game-test/closed-under-addition,
 :seed 43}
{:iterations 156217,
 :msec 10249,
 :var #'clojurebreaker.game-test/closed-under-addition,
 :seed 42}
```

Behind the scenes, `test.generative` has created a future per processor on your local hardware. Each future runs the tests as many times as possible in ten seconds. The output then shows how many iterations ran on each thread, how much wall-clock time each thread took to finish, and the random seed used to generate the data for each var that was tested.

`test.generative` exposes several dynamic vars to customize a test run. You can bind `test/*msec*` to change the length of the test run, or you can bind `test/*cores*` to change the number of cores utilized by the test run. You can also bind `test/*verbose*` to have all the inputs printed to `*out*` during the test run.

When a Spec Fails

When a spec fails, `test.generative` provides additional output to assist you in reproducing the problem. Let's write a spec that will fail by asserting that the sum of two numbers is less than either:

```
clojurebreaker/snippets.clj
(defspec incorrect-spec
  +
  [^long a ^long b]
  (assert (< a %))
  (assert (< b %)))
```

If you run `incorrect-spec`, it will fail quickly with an error like this:

```
{:form ('clojurebreaker.game-test/incorrect-spec
        -5025562857975149833 -5843495416241995736),
 :iteration 0,
 :seed 42,
 :error "Assert failed: (< a %)",
 :exception #<AssertionError java.lang.AssertionError:
           Assert failed: (< a %)>}
```

The `:form` key gives you the input that failed, and the `iteration` tells what iteration of the input generator caused the failure. Because the test data generation is pseudorandom with a well-known seed, you can get the failure to repeat by simply rerunning the entire test (possibly after adding logging or attaching a debugger).

If you do not want to repeat the entire run to get back to the point of a failure, you can access a collection of the forms that caused errors by calling the `failures` function:

```
(test/failures)
-> ({:form
    ('clojurebreaker.game-test/incorrect-spec
     -5025562857975149833 -5843495416241995736)}
   {:form
    ('clojurebreaker.game-test/incorrect-spec
     -5027215341191833961 -2715953330829768452)})
```

From here, it is a simple matter of collection traversal to reevaluate past failing inputs:

```
(eval (:form (first (test/failures))))
-> AssertionError Assert failed: (< a %)
```

We have now written and run specs and seen how to explore when things go wrong. Next we will put these techniques to use testing the Clojurebreaker score function.

Generative Testing the score Function

In [Programmatic Validation, on page 237](#), we created several functions that validated invariants in Clojurebreaker scoring. Now let's add them to a spec:

```
clojurebreaker/test/clojurebreaker/game_test.clj
(defspec score-invariants
  game/score
  [{:tag `random-secret} secret
   {:tag `random-secret} guess]
  (assert (scoring-is-symmetric secret guess %))
  (assert (scoring-is-bounded-by-number-of-pegs secret guess %))
  (assert (reordering-the-guess-does-not-change-matches secret guess %)))
```

This spec shows one new bit of defspec semantics: the use of the syntax quote (`'`) character. The syntax quote on `random-secret` indicates that the generator function is a custom one written by us. There are two rules for quoting names in an argument spec:

- If a name is not syntax-quoted, then that name is interpreted against the built-in generators from the namespace `clojure.test.generative.generators`. These names happen to be shadow types and factory names from `clojure.core`, which is why the built-in generators look like type hints.
- If a name is syntax-quoted, then the name is interpreted by the rules of the namespace the defspec resides in. This allows you to use all the normal namespace tools to manage your own generators.

And now, the moment of truth. When we run the spec, do the invariants actually hold?

```
#'clojurebreaker.game-test/score-invariants
-> [#<core$future_call$reify__5684@590e130c: :pending>
    #<core$future_call$reify__5684@2b04a681: :pending>]

;; ten seconds pass
{:iterations 1794, :msec 10002, :seed 42,
 :var #'clojurebreaker.game-test/score-invariants}
{:iterations 1787, :msec 10001, :seed 43,
 :var #'clojurebreaker.game-test/score-invariants}
```

Looks good. Can you think of other interesting invariants to test?

At this point, we have only scratched the surface of `test.generative`. And, while we like the separation of input generation, execution, and validation, it is certainly possible to use more familiar unit testing, TDD, and BDD approaches in Clojure. If you are interested in these, you should check out the following libraries:

- `clojure.test`¹ is a modest unit testing library that is built into Clojure's own test suite.
- `lazytest`² is a generic library that supports different testing styles atop a few generic representations of tests.
- `Midje`³ emphasizes readable tests, while allowing for both top-down and bottom-up testing.

1. <http://clojure.github.com/clojure/clojure.test-api.html>
 2. <https://github.com/stuartsierra/lazytest>
 3. <https://github.com/marick/Midje>

Many of the ideas in `test.generative` are inspired by QuickCheck,⁴ a testing library originally written in Haskell but now ported to many languages. If you find `test.generative` interesting, you should definitely check out QuickCheck, which has a long development history and many capabilities not yet found in `test.generative`.

Now that we have some confidence in our scoring function, let's see about running a game of Clojurebreaker on the Web.

10.4 Creating an Interface

With a solid foundation, introducing an interface and a playable version of the game should be a breeze. Let's start with a basic web application. The *noir* web framework will serve as a nice base for us to build our application. Creating a new noir project is easy.

First, you will need to install the lein-noir plug-in to leiningen:

```
$ lein plugin install lein-noir 1.2.0
```

Now we can generate our application and launch it:

```
$ lein noir new clojurebreaker
$ cd clojurebreaker
$ lein run
```

Point your browser to <http://localhost:8080>. You will get the default noir landing page, which isn't interesting to you at the moment, but it will ensure that you have things set up correctly.

It's Finally Time for Some State

This is where we will introduce our one and only piece of state. Since we want multiple players to be able to enjoy our game at once and we want to be able to maintain the game's solution between requests, we will need to store this value somewhere. The great part is that we don't have to manage it in our code. We can just put it in the browser's session, leaving our application code free of state management.

Let's start by opening `src/clojurebreaker/views/welcome.clj`. Change the `/welcome` page to just `/`. We need a way to put something in the session if it isn't already there. Luckily, noir has `session/put!` and `session/get`. Let's use them.

```
(defpage "/" []
  (when-not (session/get :game)
    (session/put! :game (.nextInt (java.util.Random.) 1000000))))
```

4. <http://en.wikipedia.org/wiki/QuickCheck>

```
(common/layout
  [:p "Welcome to clojurebreaker.
      Your current game id is " (session/get :game)]))
```

If you refresh your browser, you will now see that you have a game ID in your session. If you continue to refresh, it shouldn't change. If you clear your sessions and refresh, you should see a new game ID. We will use this technique going forward, but we won't need a randomly generated ID anymore. That was just to demonstrate that the session store is working properly.

Create a new file named `game.clj` in `src/clojurebreaker/models`. Here we will put the function we need to generate a new game secret and return it to the view.

```
(ns clojurebreaker.models.game)

(defn create []
  (vec (repeatedly 4 (fn [] (rand-nth ["r" "g" "b" "y"]))))))
```

Let's give this a try in the REPL just to make sure it is producing what we need.

```
(in-ns 'clojurebreaker.models.game)
clojurebreaker.models.game=> (dotimes [_ 5] (println (create)))
| [g y g b]
| [g r r r]
| [r y g r]
| [b y y b]
| [b g r g]
-> nil
```

This is everything we need to start our game. We just need to go back to our view and wire up the code. Remember that `test.generative` can also create your game secret, so feel free to use it instead of the earlier function. This just demonstrates another way of creating a secret.

```
(defpage "/" []
  (when-not (session/get :game)
    (session/put! :game (game/create)))
  (common/layout
    [:p "Welcome to clojurebreaker.
        Your current game solution is " (session/get :game)]))
```

Also, don't forget to require `clojurebreaker.models.game` as `game` in your namespace declaration. When you refresh your browser, you will see your game solution printed. If the solution is still an ID, you will need to clear your browser's session data and refresh. Try opening a different browser and visiting the application. You should see a different game solution.

The Player Interface

Now that we have game state, we need an interface for the players. Let's start by creating a game board. We will need to add the `hiccup.form-helpers` namespace into our namespace declaration with the rest of the `hiccup` imports. We will now take advantage of `noir`'s `defpartial` macro.

```
(defpartial board []
  (form-to [:post "/guess"]
    (text-field "one")
    (text-field "two")
    (text-field "three")
    (text-field "four")
    (submit-button "Guess")))
```

The `defpartial` macro is quite useful here, because it is used just like a regular Clojure function. This will come in handy later when we wire in scoring. Give it a try at the REPL:

```
clojurebreaker.models.game=> (in-ns 'clojurebreaker.views.welcome)
clojurebreaker.views.welcome=> (board)
| "<form action=\"/guess\" method=\"POST\">
|   <input id=\"one\" name=\"one\" type=\"text\" />
|   <input id=\"two\" name=\"two\" type=\"text\" />
|   <input id=\"three\" name=\"three\" type=\"text\" />
|   <input id=\"four\" name=\"four\" type=\"text\" />
|   <input type=\"submit\" value=\"Guess\" /></form>"
```

Now we just need to wire it in:

```
(defpage "/" []
  (when-not (session/get :game)
    (session/put! :game (game/create)))
  (common/layout (board)))
```

Since we have verified that the session bits are working, we can just remove their display and replace them with the actual game. We now have a way for users to guess. Well, at least we have a way for users to type into a browser. Let's wire up the server side.

Back in the view, we need to create another page to respond to the post to `/guess`. Here we will need to do the following:

- Accept the four inputs and send them to the scorer.
- Determine whether the player has won the game (four exact matches). If so, congratulate them on winning and add a button to start a new game.
- If the user has not won, return the number of exact and unordered matches, as well as the last set of inputs, and display them to the user.

To score the request, we need to add the game-scoring functions we created earlier in the chapter. Let's add exact-matches, unordered-matches, and score to our model. With our scoring functions in place, we can wire up our post handler and complete the first phase of the game.

clojurebreaker/src/clojurebreaker/views/welcome.clj

```
(defpage [:post "/guess"] {:keys [one two three four]})
  (let [result (game/score (session/get :game) [one two three four])]
    (if (= (:exact result) 4)
      (do (session/remove! :game)
          (common/layout
            [:h2 "Congratulations, you have solved the puzzle!"]
            (form-to [:get "/"]
                      (submit-button "Start A New Game")))))
      (do (session/flash-put! result)
          (render "/" {:one one
                       :two two
                       :three three
                       :four four
                       :exact (:exact result)
                       :unordered (:unordered result)}))))))
```

OK, so this is a bit of a REPL-full. Here we are able to determine whether the player has won the game and handle it appropriately. There are a couple of new things pulled in from noir here that haven't been introduced yet, though. The `:keys` destructuring after `defpage` is dealing with the arguments passed in from the browser. `session/remove!` does exactly what you think it might do. The interesting bits start with `session/flash-put!`. This will add something to the session for the request immediately following to consume, and then it will disappear. It is similar to the Ruby on Rails flash method. Finally, the `render` function calls the route with the arguments that follow.

With just a few modifications, we will be ready to give the game a try. Our `/` route needs to be able to accept the arguments that it is passed via the call to `render` that we just wrote. Let's fix that. First modify the board partial to accept and destructure arguments passed to it and render results:

clojurebreaker/src/clojurebreaker/views/welcome.clj

```
(defpartial board [{:keys [one two three four exact unordered]}]
  (when (and exact unordered)
    [:div "Exact: " exact " Unordered: " unordered])
  (form-to [:post "/guess"]
    (text-field "one" one)
    (text-field "two" two)
    (text-field "three" three)
    (text-field "four" four)
    (submit-button "Guess"))))
```

Here we have changed the rendering of the board to respond to the player's guesses. We will display the results of the guess and place the player's previous guess back onto the board so that they can modify it and continue. Experiment with this at the REPL to see what output is yielded given certain inputs:

```
clojurebreaker.views.welcome=> (board {:one "r"
                                       :two "b"
                                       :three "y"
                                       :four "g"})

| "<form action=\"/guess\" method=\"POST\">
|   <input id=\"one\" name=\"one\" type=\"text\" value=\"r\" />
|   <input id=\"two\" name=\"two\" type=\"text\" value=\"b\" />
|   <input id=\"three\" name=\"three\" type=\"text\" value=\"y\" />
|   <input id=\"four\" name=\"four\" type=\"text\" value=\"g\" />
-> <input type=\"submit\" value=\"Guess\" /></form>"

clojurebreaker.views.welcome=> (board {:one "r"
                                       :two "b"
                                       :three "y"
                                       :four "g"
                                       :exact 2
                                       :unordered 0})

| "<div>Exact: 2 Unordered: 0</div>
|   <form action=\"/guess\" method=\"POST\">
|     <input id=\"one\" name=\"one\" type=\"text\" value=\"r\" />
|     <input id=\"two\" name=\"two\" type=\"text\" value=\"b\" />
|     <input id=\"three\" name=\"three\" type=\"text\" value=\"y\" />
|     <input id=\"four\" name=\"four\" type=\"text\" value=\"g\" />
-> <input type=\"submit\" value=\"Guess\" /></form>"
```

Just one minor change to the / page definition, and we are ready to start playing!

```
clojurebreaker/src/clojurebreaker/views/welcome.clj
(defpage "/" {:as guesses}
  (when-not (session/get :game)
    (session/put! :game (game/create)))
  (common/layout (board (or guesses nil))))
```

Here we are just accepting the arguments from render so that we can pass them along to the board partial. Since board works like a standard Clojure function, this process is trivial.

There is just one more thing to do before we can take the game for a spin. We need to add math.combinatorics to the project.clj file in our noir project.

```
clojurebreaker/project.clj
```

```
(defproject clojurebreaker "0.1.0-SNAPSHOT"
  :description "Clojurebreaker game for Programming Clojure 2nd Edition"
  :dependencies [[org.clojure/clojure "1.3.0"]
                 [org.clojure/math.combinatorics "0.0.1"]
                 [org.clojure/test.generative "0.1.3"]
                 [noir "1.2.0"]]
  :main clojurebreaker.server)
```

Rerun `lein deps` and `lein run`, and you can now play Clojurebreaker to your heart's content.

10.5 Deploying Your Code

Now that you have a complete application, it's time to put it somewhere for the world to enjoy! Deploying an application to production can be a long and arduous task that involves adding code or moving parts to your application. Luckily, we are going to skip all of those steps and just get to the point. There is a service available that fully supports Clojure application deployment and makes it drop-dead easy. They call themselves Heroku.⁵

There are a few initial steps to getting an account set up at Heroku, but the deployment process is as easy as checking code into a source control repository. In fact, that is all you have to do. A simple git push is how you deploy.

To get to the deployment step, you will need to sign up for an account at Heroku. It is fairly straightforward and is completely free. Heroku does have paid offerings, but this example should not cost you a dime. After signing up, you will need to create an application on the Heroku platform.

The Procfile

Heroku uses a special file in your program to let it know how the program should be started. It is called the Procfile. Add this to the root of the project.

```
web: lein run
```

If you want to ensure that things are set up properly, you can install the `foreman` gem and run `foreman start`. If that successfully starts your application, you should have no trouble on Heroku.

```
foreman start
```

```
14:35:28 web.1 | started with pid 34538
14:35:33 web.1 | Starting server...
```

5. <http://heroku.com>

```

14:35:33 web.1 | 2011-12-09 14:35:33.042:INFO::Logging to STDERR
14:35:33 web.1 | Server started on port [5000].
14:35:33 web.1 | You can view the site at http://localhost:5000
14:35:33 web.1 | #<Server Server@63ce15f6>
14:35:33 web.1 | 2011-12-09 14:35:33.044:INFO::jetty-6.1.26
14:35:33 web.1 | 2011-12-09 14:35:33.070:INFO::Started SocketConnector@0.0.0.0:5000

```

The Heroku Library

Heroku provides a nice way to programmatically interact with its platform. The only gotcha here is that it requires some additional setup if you don't already have it. You will need the Ruby programming language along with RubyGems. Your systems package manager should be able to provide you with these items. If not, you can always visit <http://ruby-lang.org>.

Installing the Heroku library is done via RubyGems:

```
gem install heroku
```

Git

This will provide a heroku command that you will use for all of your Heroku-based interactions.

The next step is to create a git repository inside of your Clojurebreaker code-base. Again, your system's package manager should be able to provide you with a suitable version of git. If not, you can visit <http://git-scm.org>.

Once you have git installed, simply run git init in the root of the clojurebreaker application:

```
git init
-> Initialized empty Git repository in ~/clojurebreaker/.git/
```

This will create the initial git shell for your application. Next you need to locally commit your code:

```
git add .
git commit -m "Initial commit"
```

```

[master (root-commit) dd4f8a8] initial commit
13 files changed, 65860 insertions(+), 0 deletions(-)
create mode 100644 .gitignore
create mode 100644 project.clj
create mode 100644 resources/public/css/reset.css
create mode 100644 src/clojurebreaker/game.clj
create mode 100644 src/clojurebreaker/models/game.clj
create mode 100644 src/clojurebreaker/server.clj
create mode 100644 src/clojurebreaker/views/common.clj
create mode 100644 src/clojurebreaker/views/welcome.clj

```

Housing Your Application

At this point, you have staged your code locally to be pushed up to Heroku. Now it's time to create a Heroku application to house your program:

```
heroku create --stack cedar
```

```
Creating freezing-waterfall-3937... done, stack is cedar
http://freezing-waterfall-3937.herokuapp.com/ |
git@heroku.com:freezing-waterfall-3937.git
Git remote heroku added
```

The `--stack` argument specifies the platform on Heroku that supports Clojure applications. The application will get a random name provided by Heroku with a URL that you can visit to see your running program. You can rename this later if you want.

The Deployment

The last step is to run `git push`, and Heroku will take care of the rest.

```
git push heroku master
```

```
Counting objects: 24, done.
Delta compression using up to 4 threads.
Compressing objects: 100% (15/15), done.
Writing objects: 100% (24/24), 230.67 KiB, done.
Total 24 (delta 0), reused 0 (delta 0)
```

```
-----> Heroku receiving push
-----> Clojure app detected
-----> Installing Leiningen
      Downloading: leiningen-1.5.2-standalone.jar
      Downloading: rlwrap-0.3.7
      Writing: lein script
-----> Installing dependencies with Leiningen

... Output elided ...

-----> Discovering process types
      Procfile declares types -> web
-----> Compiled slug size is 12.8MB
-----> Launching... done, v4
      http://stormy-water-3888.herokuapp.com deployed to Heroku
```

You can visit your application using the `heroku` command as well:

```
heroku open
```

This will open a browser and send you right to your deployed application. It might take some time to load on the first visit, but subsequent requests will

get faster. You can find an additional Heroku/Clojure tutorial on Heroku's dev center at <http://devcenter.heroku.com/articles/clojure-web-application>.

Have fun with your new Clojure web application. Make it better, give it some personality, and share it with the world!

10.6 Farewell

Congratulations. You have come a long way in a short time. You have learned the many ideas that combine to make Clojure great: Lisp, Java, functional programming, and explicit concurrency. And in this chapter, you saw one (of a great many) possible workflows for developing a full application in Clojure.

We have only scratched the surface of Clojure's great potential, and we hope you will take the next step and become an active part of the Clojure community. Join the mailing list.⁶ Hang out on IRC.⁷ The Clojure community is friendly and welcoming, and we would love to hear from you.

6. <http://groups.google.com/group/clojure>

7. [#clojure on irc.freenode.net](#)